What Happens to a Monitor's Accuracy When You Train Against It

Abraham Yeung[*]
Stanford University

Anagha Ramaswamy
Stanford University

Abstract

A linear probe on a language model's hidden states predicts whether the model's next answer will be correct. On held-out prompts it ranks a correct answer above an incorrect one 98% of the time. The probe we train against is a second fit of the same kind, on temperature-1 answers like those reinforcement learning samples. Used as the reward, it lowers true accuracy by 31 percentage points.

The reward probe's AUROC does not show this for 30 to 40 training steps. Scored at every checkpoint on that checkpoint's fresh answers, it first falls at step 40, or first moves at step 50 if each answer is graded by its last answer block rather than its first, although task accuracy is already significantly lower at step 10. A frozen LLM judge that was never trained against ends the run with the pooled AUROC it started with while accuracy falls from 54% to 23%, although among answers that use the given numbers correctly its AUROC falls. A monitor's accuracy statistics describe the monitor together with the population it scores, and training against one monitor moves the population that every monitor scores. A stable pooled AUROC can average a loss on hard cases with a growing share of easy ones, and it does not say how many answers are correct. The share of answers above a threshold frozen at the start moves without labels, by step 10 for the probe and later for the judge. For the probe that movement is partly guaranteed by the objective, and neither has been calibrated against matched benign training.

Held-out accuracy also did not order our two probe rewards by how safe they were to train against. Of three rewards trained under one recipe, each reading a frozen copy of the starting model or only the text, a probe on that copy's activations with 39 surface features of the text regressed out (AUROC 0.958) left the most accuracy (0.168). The same probe without that step read higher (0.978) and left 0.051. The residualised reward was also the weaker on the model it read, and it penalised the surface pattern rather than ignoring it, so this is a counterexample, not evidence that removing surface information makes a reward safe. A monitor built only from those 39 text features, which read lowest (0.900), drove accuracy to exactly zero. Nothing in the objective rewarding evading the monitor. The policy is doing what it was asked.

1 Introduction

A probe is a small classifier trained on a model's internal activations to predict some property of the model's behaviour. Directions and detectors read from a model are increasingly used for more than analysis. Refusal directions steer behaviour when ablated or added (Arditi et al., 2024), truth probes aim to recover what a model knows apart from what it says (Burns et al., 2023), and lie detectors,

[*]Correspondence: ayeung16@stanford.edu. Code and the regeneration script: https://github.com/Abraham-y/reader-writer-probe-rl.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Interpretability as a Science (InterpScience).

black-box (Pacchiardi et al., 2024) or probe-based (Taufeeque et al., 2026), have been proposed as training signals. Placing a probe inside the training loop and optimising against it is less well understood than evaluating the probe from outside the loop.

Reward overoptimisation shows that optimising too far against a learned proxy reward degrades the true objective, with no adversarial intent required (Gao et al., 2023; Coste et al., 2024). Almost no non-trivial proxy is unhackable in principle (Skalse et al., 2022), and gaming learned on easy targets can generalise to reward tampering (Denison et al., 2024). Monitor evasion shows that monitors of a model's chain of thought (Baker et al., 2025) or of its activations (Gupta and Jenner, 2025; Bailey et al., 2024) can be evaded when they are optimised against, though with enough KL regularisation and a strong enough probe penalty, training can instead yield honest policies (Taufeeque et al., 2026). Work on LLM evaluators reaches a related conclusion from the measurement side. Judges are usually validated by agreement with human labels (Zheng et al., 2023), agreement statistics overstate how well they discriminate (Norman et al., 2026) and hide their sensitivity to how the rubric is worded (Weng et al., 2026), and the measurement changes when the judge does (Yang et al., 2026). Reward-model benchmark scores track downstream outcomes unevenly, depending on the metric and on how the reward model is used (Frick et al., 2024; Malik et al., 2026).

Both mechanisms are established. Less is known about what a monitor's own quality statistics do while the policy it scores degrades. Baker et al. (2025) report that a chain-of-thought monitor's recall falls to near zero once the policy is optimised against it, measured against a programmatic check for one class of hack: whether the tests still pass once the agent's edits to them are reverted. They note that without such ground truth, one could not tell that the agent was misaligned. That case is evasion, because the monitor entered training as a penalty. In our setting the monitor enters as a positive reward, so there is nothing to evade, and Countdown provides exact ground truth for every answer at every checkpoint. This makes it possible to measure what a monitor's discrimination does while the policy is complying with it. In our run, the monitor's AUROC falls 30 to 40 steps after task accuracy. This is consistent with Beigi et al. (2026), who find that a policy's answers to direct questions about its proxy reward forecast later reward hacking while the visible hack rate is still low, and that activation directions for the same capability emerge before sustained hacking. The information exists, but it is not in the monitor's AUROC.

This matters for probes in particular. Statistics such as AUROC and precision are properties of a classifier together with a population, yet probe AUROC is usually reported as if it belonged to the probe alone. Belinkov (2022) argues that probe accuracy does not license the claim that the model uses the probed property. Probe accuracy also does not license a claim about the probe, because training against the artifact changes the population on which the number was measured. The judge in §4.2 is frozen, and its pooled AUROC ends where it began while the policy it scores loses most of its accuracy.

We work with Qwen2.5-0.5B trained on Countdown arithmetic (Gandhi et al., 2024) with RLOO (Ahmadian et al., 2024). The task has an exact verifier, so a learned scorer's divergence from ground truth can be measured directly, without confounds from preference modelling. Four different scorers are referred to in this paper as "the monitor", and the same checkpoints are sampled and graded under four protocols. Table 1 names each, and every figure and table below states which it uses.

Contributions.

1. Held-out quality did not order our probe rewards by how safe they were to train against. Our best monitor, the trace-final probe, reaches AUROC 0.982. The probe trained against in our first runs is a second fit, on temperature-1 answers; its direction has cosine 0.192 with the first's, and it costs 31 pp when made the reward. The controlled comparison is between three rewards trained under one recipe, each reading hidden states from a frozen copy of the starting model, or only the text. On answers cut as in training, a probe with surface features regressed out (arm A) reads 0.958 and left 0.168 accuracy, while the same probe without that step (arm R) reads 0.978 and left 0.051, 11.6 pp less (CI $[9.6, 13.8]$). A reward built only from text features (arm B) read lowest (0.900) and drove accuracy to exactly 0.0000. Arm A's reward was also the weaker on the model the arms read and penalised the surface pattern a text-only reward favours, so we do not attribute the gap to removing surface information. Single-seed runs give counterexamples, not a trend. (§4, §4.3)

(a) the four monitors

name	what it reads	what it was fit to	AUROC	how it is used here
trace-final probe	L16 `</think>`	headline answers, first block	0.982	read-only; never a reward
reward probe	L16 `</think>`, *another* fit	T=1 answers, last block	0.810 / 0.945	the RL reward; scored in Fig. 1
surface model	39 text features, no activations	headline answers, first block	0.926 / 0.925	free baseline; arm B's reward
LLM judge	problem + equation (text)	not fit; a fixed prompt	0.785	bystander; never optimised against

(b) the four evaluation protocols, all on the same 406 prompts, at most 1,024 new tokens

name	answers per prompt	sampling	block graded	where
headline	16	T=0.6, top-p 0.95, top-k 20	first	§3; §4 accuracies
arm	8	T=1, top-p 1	first	arms A, B, R and C (Table 2)
probe ladder	8, fresh per checkpoint	T=1, top-p 1	both	§4.1, Fig. 1, Tables 3, 4
judge ladder	8, fresh per checkpoint	T=1, top-p 1	first	§4.2, Table 5

Table 1: Who is who. **(a)** Where two AUROCs are given they are two fits of the same features under different label rules and splits, and are *not* interchangeable: the reward probe reads 0.810 on its own protocol (last block, on answers to 300 prompts from the RL training pool, none of them among the 406) and 0.945 on the population the reward arms share (first block); the surface model reads 0.926 under the `GroupKFold` of §3 and 0.925 as the shipped arm-B artifact. The judge is `Qwen2.5-7B-Instruct`; its 0.785 is on `runA`'s step-0 judge-ladder answers; on the probe ladder's draw from the same checkpoint the reward probe reads 0.919. Table 2's AUROC columns put all four rewards on one population; §4.3 also reads arms A, B and R on C_{SFT}'s answers. **(b)** "First block" grades the first `<answer>` block in an answer, "last block" the last. During RL, sampling stopped at the first `</answer>`, so the verifier only ever graded the first block; every evaluation here samples without that stop, which is why later blocks exist at all. The two ladders are independent draws from the same checkpoints, and their first-block accuracies agree to within 0.016 at every checkpoint. The headline protocol's settings are the sampler's defaults; its output files do not record them.

2. A monitor's AUROC registered the decline late or not at all. Task accuracy on the probe-as-reward run is significantly lower by step 10. At our 10-step checkpoint resolution, the reward probe's AUROC first falls at step 40 under first-block labels, the block the verifier graded during training. Under last-block labels, the label the probe was fit to, it first moves at step 50. Its positive likelihood ratio (LR+) at the step-0 median threshold falls sooner, by step 10 under first-block labels, because inflating scores raise its false-positive rate proportionally more than its true-positive rate, though at a lower threshold it rises instead. The central case is a frozen LLM judge that was never optimised against. Its pooled AUROC ends where it began, but among well-formed answers its AUROC falls, and so does its LR+ at stricter thresholds. Training against one monitor moves the population that every monitor, including a bystander, scores, and a flat pooled AUROC can average a loss on hard cases with a growing share of easy ones. The share of answers above a threshold frozen at step 0 registers the inflation at step 10 without labels, 30 to 40 steps before the AUROC falls (0 to 20 against our imperfect benign band). For the probe that early movement is partly guaranteed by the objective, and using the statistic as an alarm would require a matched benign control, which has not been run. (§4.1, §4.2)

2 Setup

We use Qwen2.5-0.5B on Countdown. Given 3–4 integers and a target, the model must produce an equation using each integer exactly once, checked by an exact rule-based verifier. C_{SFT} is a supervised checkpoint. C_{outcome} is 100 RLOO steps against the verifier, which raises first-block accuracy on our 406 evaluation prompts from 0.290 to 0.550 (headline protocol, Table 1b). The evaluation set is 406 procedurally generated problems with no overlap against the RLOO training pool; the overlap filter removed mostly three-number problems, so 61% of the 406 use four numbers. AUROC here is the probability that a randomly chosen correct answer scores above a randomly chosen incorrect one, so 0.5 is chance. Appendix C gives the probes, hyperparameters and estimator.

3 The probe is a good detector

A trace-final probe on layer-16 `</think>` states reaches AUROC 0.982 on C_{outcome}'s headline answers, held out by prompt, meaning it is scored on prompts it was not fit on. As a best-of-16 selector it captures 88.8% of the accuracy gain available to any selector (CI [78.9%, 96.5%]). Free baselines come close, which matters in §4.3. `</think>` position alone scores 0.915, and a 39-feature

surface model scores 0.926. As a selector the probe beats both free baselines by +3.20 pp of selection accuracy. The probe is a good detector that partly relies on a cheap surface correlate, and it is not only a length detector. Appendix D gives the baselines and the length controls.

The probe evaluated here is not the probe trained against below, and the two share little direction (cosine 0.192). To match the temperature-1 answers the policy samples during RL, rather than the trace-final probe's temperature-0.6 answers, the reward probe was fit separately, on temperature-1 answers to 300 prompts from the RL training pool. None of those prompts is among the 406 evaluation prompts. Its labels grade the last answer block of answers sampled without the stop at `</answer>` that RL training used, and its held-out AUROC on that protocol is 0.810. On C_{outcome}'s headline answers graded by the first block it reads 0.951 over all 406 prompts (0.945 on the half used in Table 2). The separate fit was a choice, not a requirement: arm R of §4.3 trains against a probe fit on temperature-0.6 answers, at cosine 0.779 with the trace-final probe.

4 Using the probe as the reward destroys the task

The reward probe's score is now the quantity the model is trained to maximise. The reward is its $P(\text{correct})$ at `</think>`, read from the current policy's hidden state; the probe is fixed, and the copy of the policy it reads is refreshed each round (except on one round in ten, Appendix C). The verifier still runs, but only to log true accuracy. It does not enter the gradient. Training runs 100 RLOO steps. Against the run that produced C_{outcome} it keeps the learning rate, KL and entropy coefficients and sampling, but uses group size 8 rather than 16, weight decay 0.01 rather than 10^{-4}, and gradient clipping at 1.0 rather than none (Appendix C).

We ran this from two starting points.

`runA` Starts from C_{outcome}, so the probe is in-distribution at step 0.

`runB` Starts from C_{SFT}, so the probe is out-of-distribution at step 0.

Both probe-reward runs lose most of their accuracy. On the 406 evaluation prompts, under the headline protocol (Table 1b), `runA` falls to 0.236. That is −31 pp from C_{outcome}'s 0.550. `runB` falls to 0.073, which is −22 pp from C_{SFT}'s 0.290. Both runs ended training with a mean probe score of about 0.99.

The trained policy writes answers that have the form of reasoning without its content. Under the headline protocol, a template score counting which of seven reasoning phrases appear (such as "let me", "step by step", "therefore" and numbered steps) rises 1.81 → 3.23 from C_{outcome} to `runA` and 3.24 → 4.32 from C_{SFT} to `runB`. Its correlation with first-block correctness falls from r=+0.643 to +0.068 and from +0.168 to +0.023. The probe continues to score these answers as correct. §4.3 gives a verbatim example of the same pattern under a text-only reward.

4.1 The probe's AUROC does not fall for 30 to 40 steps after accuracy does

The probe's own accuracy is scored at every checkpoint to determine when it registers that the policy trained against it is getting worse at the task.

This can be measured precisely here because the monitor is a correctness probe on a verifiable task. Gao et al. (2023) replace human labels with a synthetic gold reward model, itself a learned model, because labelling many checkpoints by hand is too expensive; here the verifier is exact. We take all eleven checkpoints of `runA`. At each checkpoint we sample 8 fresh answers per prompt at temperature 1 from that checkpoint (the probe ladder of Table 1b) and score them with the frozen reward probe. We compare the scores against two labels: the first block, which is what the verifier graded during training, and the last block, which is what the probe was fit to predict. The probe's weights, the prompts and the sampling settings are held fixed; the answers, and the hidden states through which the probe reads them, come from each checkpoint in turn. Figure 1 shows the result, and Tables 3 and 4 in Appendix E give every checkpoint.

Task accuracy falls first. By step 10, first-block accuracy is 5.2 points below its step-0 value of 0.546 (95% CI [−6.9, −3.7], paired over prompts), and last-block accuracy is 6.4 points below its 0.372 ([−8.2, −4.6]). Counting the 1% of answers with no locatable `</think>` as wrong, the falls are 4.8

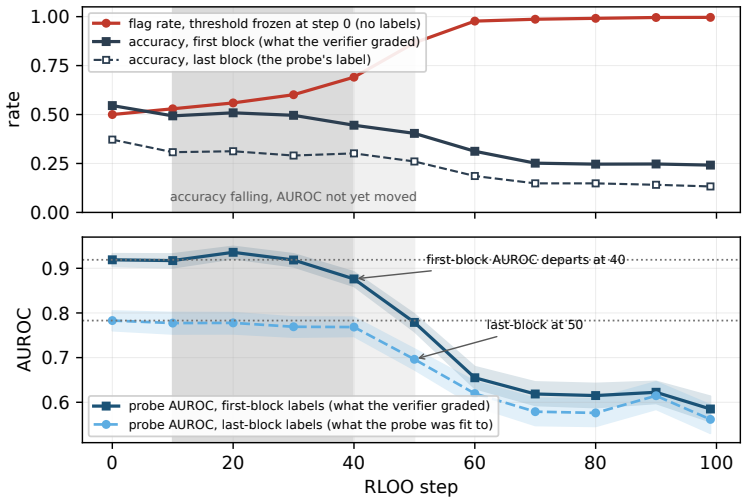


Figure 1: The monitor's AUROC does not register the decline for 30 to 40 steps. `runA`, initialised from C_{outcome}, under the probe-ladder protocol: at each checkpoint, 8 fresh answers per prompt on the 406 prompts, sampled at temperature 1. **Top:** accuracy under both grading rules is significantly lower by step 10, while the fraction of answers above a threshold frozen at step 0 rises. The shaded span runs from the first significant fall in accuracy to the first significant fall in AUROC, at step 40 (dark) under first-block labels and step 50 (light) under last-block labels. Dotted lines in the bottom panel mark each AUROC's step-0 value. **Bottom:** the frozen reward probe's AUROC under each label, with 95% prompt-clustered intervals. Both panels are computed from the same data as Tables 3 and 4.

and 6.1 points. By step 40 the falls are 10.0 and 7.0 points. Under first-block labels the probe's AUROC starts at 0.919, rises slightly at step 20 ($+0.017$, nominal $p = 0.022$, which does not survive correction), and first falls at step 40, by -0.043 ($p < 0.001$). Under last-block labels it starts at 0.783, does not differ detectably from step 0 through step 40 (all $p \geq 0.22$), and first moves at step 50, by -0.087 ($p < 0.001$). Either way the AUROC does not differ detectably from its start for 30 to 40 steps after accuracy has fallen, at our 10-step checkpoint resolution, and this ordering needs no argument about the probe, since accuracy is measured by the verifier. It is not a matter of power: the two changes have similar bootstrap standard errors, about 0.01, and at step 10 accuracy has moved by six of them and AUROC by less than one. The two labels differ because answers sampled without training's stop at `</answer>` run on into further answer blocks the policy would never have written in training, and those later blocks are more often wrong.

A flat AUROC also means what it says. Until it moves, the probe still ranks the policy's correct answers above its incorrect ones about as well as it did at step 0. What AUROC cannot report is that there are fewer correct answers to rank. Part of the reason it is flat is the metric rather than the policy. AUROC depends only on the order in which the probe ranks a checkpoint's answers. Between steps 0 and 40 the mean probe score inflates $0.473 \to 0.677$. Read through the step-40 checkpoint, the step-0 answers score lower than at step 0 (0.377 against 0.473) and lose little first-block AUROC (0.913 against 0.919). The inflation is therefore in the answers the policy now writes, not in how the probe reads old ones. A shift of the whole score distribution, of any size, leaves a rank-based statistic unchanged if the order within it holds, so the flatness before the AUROC moves follows from what AUROC measures and is not specific to the probe. Precision at a frozen threshold is confounded with the falling share of correct answers. The prevalence-invariant LR+ at the step-0 median threshold does move early: $7.00 \to 5.63$ by step 10 under first-block labels ($p \approx 0.01$), and $2.41 \to 1.91$ by step 30 under last-block labels ($p < 0.001$). Inflating scores raise the false-positive rate proportionally more than the true-positive rate (FPR $0.117 \to 0.161$, TPR $0.819 \to 0.907$ by step 10). This depends on the threshold: at the step-0 25th percentile, LR+ under first-block labels instead rises at step 10 ($2.03 \to 2.41$). Appendix B works through these.

Statistics that need no labels can register the inflation. Each of the statistics above requires correctness labels, with which the accuracy drop could be measured directly. Score saturation is one label-free

statistic: the share of answers above 0.95 rises from 5.2% at step 0 to 38.8% by step 40, while on the benign ladder of Appendix B the same probe keeps it below 9% at every checkpoint. The flag rate is a finer version. We freeze a threshold at step 0, its median score (0.540), and record the share of answers that clear it. Under attack this share rises monotonically from 0.500 (0.529, 0.559, 0.601 and 0.691 at steps 10 to 40, and 0.996 by step 99). It moves at step 10, by $+0.029$ $[+0.012, +0.048]$ ($p < 0.01$), whereas the AUROC first changes at step 40 or 50. At a lower frozen threshold it also moves at step 10, but downwards ($0.750 \to 0.699$ at the step-0 25th percentile): the score distribution widens before it shifts. The two statistics, computed from the same probe on the same answers, are therefore separated by 30 to 40 steps.

Some of that separation is expected: the flag rate is a threshold-crossing rate of the probe score, whose mean gradient ascent raises by construction. Nothing in the objective requires the score to inflate for 30 to 40 steps while its ranking holds, and the ordering between accuracy and AUROC does not share this circularity, since accuracy comes from the verifier. For a label-free signal, the non-circular case is the judge of §4.2, whose flag rate also moves.

Turning the ordering into a detector requires a benign band from a matched ladder of ordinary verifier-reward checkpoints, and our benign control ladder has five defects (Appendix B). Against it, the attacked run's flag rate first exceeds the band at step 30, narrowly, so it leads the AUROC by 0 to 20 steps rather than 30 to 40, and it is not a calibrated alarm. The ordering itself is measured on the attacked run alone.

4.2 A frozen judge keeps its pooled AUROC while the policy degrades

The probe was also the training target, so its flat AUROC, like its flag rate, could reflect a change in how the probe relates to the answers rather than a change in the population. A monitor that cannot change, and was never optimised against, separates the two cases when it scores the same run: if its AUROC also stays flat, the flatness cannot come from the monitor changing.

The monitor is an LLM judge (Zheng et al., 2023). We re-sample all eleven checkpoints of `runA` (the judge ladder of Table 1b: 406 prompts $\times$ 8 answers at temperature 1, 35,728 answers in total). This is a second, independent draw under the probe ladder's sampling settings, graded by the first answer block. Every answer is scored twice, once with the exact verifier and once with `Qwen2.5-7B-Instruct`, which is shown the problem and the proposed equation and asked whether the equation is correct. The judge's score is $P(\text{YES})$ renormalised over the verdict tokens. Appendix A gives the prompt and decoding settings. The judge is imperfect. At step 0 it reaches precision 0.746 at its own median threshold ($P(\text{YES}) \geq 0.9991$), against a true accuracy of 0.544. The policy was never optimised against it.

The judge's weights and prompt are fixed. Across all eleven checkpoints it scored $4{,}651$ (problem, equation) pairs more than once; 8 cross the threshold, all within 3×10^{-4} of it, and of the 48 pairs scored at both step 0 and step 99, none do. Any change in its statistics below is therefore not a change in the judge.

Its pooled AUROC does not register the decline (Table 5). It is 0.785 at step 0 and 0.769 at step 99. The one excursion is a dip of -0.031 at step 30, across steps 20–40 while true accuracy falls $0.544 \to 0.457$; at step 50 it is 0.778. The dip's nominal p is 0.002 in Table 5 and about 0.003 pooled over bootstrap seeds, which does not reliably survive correction over the judge's 20 contrasts. About half of the dip comes from one feature of the step-0 population: step 0 has 108 answers with no `<answer>` block, which the judge scores as near-certain failures, against at most 6 at any later step. Without them the step-0 AUROC is 0.769 and the dip is -0.015 ($p = 0.18$). The first and last checkpoints do not differ detectably (-0.017, 95% CI $[-0.048, +0.014]$), though a fall as large as the probe's at step 40 is not excluded. Raw precision does fall, by 42% ($0.746 \to 0.429$), but the share of correct answers falls $0.544 \to 0.226$ over the same span, and precision at a fixed threshold moves with that share by construction. At the median threshold $\text{LR}+ = \text{TPR}/\text{FPR}$, which is prevalence-invariant, ends where it began ($2.46 \to 2.57$, ratio CI $[0.88, 1.24]$). Both of its rates fall, though: the judge passes fewer correct answers (TPR $0.686 \to 0.600$) and fewer wrong ones (FPR $0.279 \to 0.234$). We do not use lift, which is not prevalence-invariant (Appendix B). The tables report nominal p; Appendix E gives which contrasts survive correction.

The pooled AUROC hides a fall on the answers that are hard to judge. Every correct answer uses the given numbers correctly, so the pooled AUROC is a weighted mean of two AUROCs. Against

wrong answers that also use them correctly it falls from 0.703 at step 0 to 0.628 at step 99 (change CI $[-0.116, -0.033]$). Against malformed ones (wrong numbers, or no answer block), which the judge rejects easily, it goes from 0.935 to 0.956. Malformed answers grew from 36% of wrong answers to 43%; at step 0's mix, step 99's pooled AUROC would be 0.745 rather than 0.769. Pooled over all answers, at a stricter threshold, the step-0 75th percentile, the judge's LR+ falls $4.61 \rightarrow 2.49$ (ratio CI $[0.33, 0.86]$).

The judge did not change, and its pooled AUROC ended where it began while true accuracy fell $0.544 \rightarrow 0.226$, a drop of 31.7 points, or 58%. That is the problem for monitoring, because pooled AUROC is the statistic a monitor is usually validated on. Since the judge is fixed, whatever moved in its statistics moved in the population it scored. Besides the growing share of malformed answers, the surface form changed. The policy stopped using parentheses, with the share of correct answers containing none rising $0.017 \rightarrow 0.982$, and the judge is sensitive to this. For correct expressions written both with and without brackets across steps 0 and 99, it passes 0.746 of the bracketed forms and 0.548 of the bare ones ($+0.198$, 95% CI $[+0.127, +0.266]$). The judge therefore also depends on a surface correlate. This is the situation clinical epidemiology calls spectrum bias (Ransohoff and Feinstein, 1978): a test's sensitivity and specificity, measured on one spectrum of cases, do not carry over to another. In spectrum bias the study population varies for reasons external to the test, such as who is enrolled. Here training against a monitor is what moves the population, so the number an interpretability paper reports from held-out evaluation is measured on a distribution that training removes.

The frozen-threshold statistic also registers the change for the judge, though here the share of answers above the threshold decreases rather than increases, $0.500 \rightarrow 0.317$. The departure is nominally detectable at step 20 (-0.026 $[-0.047, -0.007]$, $p = 0.013$) but not at step 10 ($p = 0.069$), and it survives correction from step 40 on; its timing also depends on where the threshold is frozen. It is not monotone, reaching a minimum of 0.294 at step 80 while accuracy continues to fall. The judge was never optimised against, so this departure is not a consequence of the training objective; at the judge's step-0 rates, the fall in the share of correct answers alone predicts about 70% of it. A frozen operating point can therefore register the population shift on its own, though later than the probe's. The quantity to monitor is distance from that operating point in either direction, not a rise specifically. A two-sided anytime-valid test, built the way the anchor process of Li (2026) runs one e-process per direction, would suit it; we did not run one.

The judge is a bystander, so this is not a demonstration that a judge can be gamed. Zhou (2026) optimises a policy against its own judge and finds the errors transfer: a three-family ensemble, two of whose members were never trained against, sees its discrimination ($\mathrm{TPR} - \mathrm{FPR}$) fall from 0.31 to 0.09 while still passing 55% of hacked wrong answers. A labelled check registers that fall. Ours keeps its pooled AUROC, and at its median threshold its $\mathrm{TPR} - \mathrm{FPR}$ moves only from 0.41 to 0.37; at the step-0 75th percentile it falls from 0.30 to 0.08.

4.3 Reading higher as a monitor did not make a safer reward

The probe partly depends on surface properties of the text (§3), which raises the hypothesis that the policy degrades by learning to produce them. We test it with detectors that use only the text and with RL arms that change the reward.

The first is already in §3. A 39-feature surface model with no access to activations reaches 0.926, close to the probe's 0.982, so much of what the probe detects is also visible in the text. Its features are lengths and counts in the reasoning trace, the answer block and the text after `</think>`, and counts of 24 marker words such as "wait" and "verify". The second is three RL arms, two of them pre-registered (Appendix C). Arm A follows the logic of amnesic probing (Elazar et al., 2021), though not its method. We subtract from the activations the part that the 39 surface features predict, by least squares, refit the probe on what remains, and train against it. Arm B's reward is the text-only surface model itself. Arm R, added after review, trains against the raw probe that arm A's fitting procedure produces alongside the residualised one: the same rows, labels, prompt split and regularisation, without the subtraction. All three start from C_{SFT} and share one recipe: the same trainer, which reads hidden states from a frozen copy of C_{SFT} so that each reward is a fixed function of the text, and the same optimiser settings and evaluation. Arms A and R therefore differ only in the residualisation. Their probes and arm B's model were fit on C_{outcome}'s answers to one half of the 406 evaluation prompts, and every AUROC below is on the other half. `runB`, which also starts from C_{SFT}, appears

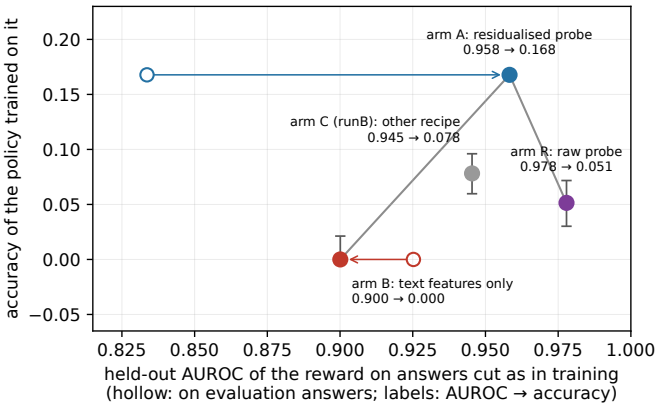


Figure 2: Higher held-out AUROC did not indicate the safer training target. Each point is one reward: its AUROC on prompts it was not fit on, read through C_{outcome} on answers cut at the first `</answer>` as in training (filled). It is plotted against the first-block accuracy of the policy after 100 RLOO steps against it, under the arm protocol (Table 1b). Hollow markers give arms A and B on evaluation answers, which run on past the first `</answer>`. Arms A, B and R share a recipe and differ only in the reward; the raw probe (R) reads highest and kept far less accuracy than the residualised one (A). Arm C (`runB`) differs from the arms in trainer and optimiser settings (§4.3), so it is shown for reference, in grey. Bars are paired prompt-clustered 95% intervals of each arm's difference from arm A, drawn at each arm's accuracy and cut at zero for arm B; the line joins the three arms on one recipe. The intervals measure evaluation uncertainty for each trained policy, not variation across training runs. Table 2 gives the same four rows.

as arm C for reference, but it differs from the arms in more than its reward. Its probe read the current policy's hidden states, it used weight decay 0.01 and gradient clipping at 1.0 where the arms used 10^{-4} and none, and its probe was fit on different answers. Arms A, B and R form the controlled comparison (Figure 2, Table 2).

Which text a reward is scored on matters for arms A and B. During RL, generation stopped at the first `</answer>`; our evaluation answers run on past it. The length of the text after `</think>`, one of the 39 features, is therefore about $1{,}160$ characters on evaluation answers and 61 on the same answers cut as in training, and its correlation with correctness changes sign ($+0.72$ to -0.20). On answers cut as in training the rewards read A 0.958, B 0.900 and R 0.978; uncut, A 0.834, B 0.925 and R 0.978. Arm B's place depends on the text. Read instead through C_{SFT}, on C_{SFT}'s own headline answers cut as in training, they read lower: A 0.721, B 0.660 and R 0.816. The raw probe reads above the residualised one on every reading, by 0.020 (CI $[0.012, 0.028]$) on answers cut as in training and by 0.095 (CI $[0.078, 0.112]$) through C_{SFT}.

Because the subtraction is a fixed linear map, arm A's reward is a probe on the activations plus a fixed term in the text features. On answers cut as in training, the activation term alone reads 0.976, the text term alone 0.094, and the two together 0.958. The text term's weights oppose arm B's on 24 of the 27 features that vary ($r = -0.56$), and on arm B's final answers it contributes -24.2 logits, against $+2.82$ on C_{SFT}'s. Arm A therefore tests a probe reward that penalises the surface pattern a text-only reward produces, not one blind to it.

A reward with no access to the model's internals drove accuracy to exactly zero, and the output keeps its form. Every answer still emits `</think>` and $3{,}244$ of $3{,}248$ contain an `<answer>` block, but only 0.12% of those blocks are arithmetic at all; Appendix F gives a verbatim example, which has the form of verification and no valid arithmetic. The reward's largest weights favour long text after `</think>` (`tail_len`, $+1.19$ in standardised units) and a long answer block (`ans_len`, $+0.62$), and penalise equals signs in the reasoning (`n_equals`, -1.11). They were fit on evaluation answers, where the text after `</think>` runs on past the answer. In training the text stopped at the first `</answer>`, so the same weights rewarded a long answer block and scored ordinary answers low: 0.12 on average on C_{SFT}'s, against 0.53 on the same answers uncut.

Arm	reward	AUROC training cut	AUROC eval. text	first-block acc. (vs. arm A)
A	surface-*residualised* probe	0.958	0.834	0.1678 (reference)
B	surface features *only*	0.900	0.925	**0.0000** (-16.78 pp, CI $[-18.97, -14.66]$)
R	the same probe, *raw*	0.978	0.978	0.0514 (-11.64 pp, CI $[-13.76, -9.61]$)
C†	raw reward probe (`runB`)	0.945	0.945	0.0782 (-8.96 pp, CI $[-10.81, -7.17]$)

Table 2: The rewards, compared. AUROCs are on the half of the 406 prompts that arms A, B and R were not fit on: C_{outcome}'s headline answers, read through C_{outcome} and graded by the first block, either cut at the first `</answer>` as in training (training cut) or as sampled for evaluation (eval. text). Only arms A and B read the text after `</think>`, so only their AUROCs differ. Read through C_{SFT}, all three arms read lower (§4.3). Accuracy is first-block at step 100, all four under the arm protocol (Table 1b). Intervals are paired over prompts, for fixed trained policies. †Arm C differs from arms A, B and R in trainer and optimiser settings as well as reward (§4.3), so only A, B and R are controlled.

The pre-registered predictions compare first-block accuracy at step 100 with the published probe-reward run, `runA`. Read as final accuracy, arm A's prediction fails: it ended 6.83 pp below `runA`, a contrast that crosses initialisation and evaluation protocol. Read as the fall from each run's start, it holds (Appendix C). Against arm R, added after review to isolate the residualisation, arm A kept $+11.64$ pp more accuracy (CI $[+9.61, +13.76]$).

Three rewards under one recipe at one seed each give counterexamples and not a relationship (Figure 2). The raw probe reads higher than the residualised one on either text, and its run ended 11.6 pp lower, with a paired interval far from zero. Arm B's place depends on the text: it reads above arm A on evaluation answers and below it on answers cut as in training, and it did worst. Residualisation lowered the probe's AUROC by 0.020 on answers cut as in training, or by 0.144 uncut. Two explanations remain open that one run per arm cannot separate. A reward that is weaker on the model it reads may simply be harder to exploit, a confound the pre-registration itself named, though arm B, the weakest on either reading, was exploited most, so this could hold at most among activation probes. And arm A's reward penalises the surface pattern directly. The intervals are over prompts for fixed trained policies; whether these reversals repeat across training seeds is untested.

Producing cheap surface structure is sufficient to cause the degradation, but is not shown to be its mechanism. It is sufficient to defeat both a monitor and the task, with no access to internals. This does not establish that it is the mechanism behind the probe reward's failure; arm A is consistent with that account but does not establish it. Which directions an optimiser can push a monitor's score along is a different question from what the monitor can detect, and held-out accuracy answers only the second.

5 Limitations

Every RL configuration is a single seed, including the checkpoint ladder on which the lag is measured. Our prompt-clustered intervals measure evaluation uncertainty for each trained policy, not variability across training runs. Training was not seeded; repeating arms A, B and R over seeds is the direct replication. These results demonstrate direction and not size. All RL experiments are at 0.5B, so how these results scale is untested. We use one task, one model family, one RL algorithm and one bystander judge. The flag rate has no matched benign control, so it is a measure of change and not a calibrated alarm.

An earlier version reported that RL makes the probe direction causally active under steering; we withdraw it and report no causal experiment on the probe's direction. The steered vector has cosine 0.163 with the probe actually optimised, not the 1.000 the claim assumed. The intervention was also applied one transformer block and 2–3 tokens downstream of the read site, and the key contrasts reach only $p = 0.063$ and $p = 0.080$. The cosine between the steered vector and the optimised direction would have caught the error; claims that optimisation made a direction causal should report it and confirm that the intervention site is the read site.

Appendix C describes how every number is regenerated.

Acknowledgments and Disclosure of Funding

This work was supported by compute credits from Modal. We thank the reviewers, whose comments led to the evaluation protocols of Table 1b and to the audit that corrected several claims in the submitted version.

References

Gandhi, K., et al. Stream of search (SoS): Learning to search in language. arXiv:2404.03683, 2024.

Ahmadian, A., et al. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. ACL, arXiv:2402.14740, 2024.

Arditi, A., et al. Refusal in language models is mediated by a single direction. NeurIPS, arXiv:2406.11717, 2024.

Burns, C., et al. Discovering latent knowledge in language models without supervision. ICLR, arXiv:2212.03827, 2023.

Pacchiardi, L., et al. How to catch an AI liar: Lie detection in black-box LLMs by asking unrelated questions. ICLR, arXiv:2309.15840, 2024.

Gao, L., Schulman, J., Hilton, J. Scaling laws for reward model overoptimization. ICML, arXiv:2210.10760, 2023.

Coste, T., et al. Reward model ensembles help mitigate overoptimization. ICLR, arXiv:2310.02743, 2024.

Skalse, J., et al. Defining and characterizing reward hacking. NeurIPS, arXiv:2209.13085, 2022.

Denison, C., et al. Sycophancy to subterfuge: Investigating reward tampering in language models. arXiv:2406.10162, 2024.

Baker, B., et al. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv:2503.11926, 2025.

Gupta, R. and Jenner, E. RL-Obfuscation: Can language models learn to evade latent-space monitors? arXiv:2506.14261, 2025.

Taufeeque, M., Heimersheim, S., Gleave, A., and Cundy, C. The obfuscation atlas: Mapping where honesty emerges in RLVR with deception probes. ICML (Oral), arXiv:2602.15515, 2026.

Ransohoff, D. F. and Feinstein, A. R. Problems of spectrum and bias in evaluating the efficacy of diagnostic tests. *New England Journal of Medicine*, 299(17):926–930, 1978.

Belinkov, Y. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, arXiv:2102.12452, 2022.

Hewitt, J. and Liang, P. Designing and interpreting probes with control tasks. EMNLP, arXiv:1909.03368, 2019.

Voita, E. and Titov, I. Information-theoretic probing with minimum description length. EMNLP, pp. 183–196, arXiv:2003.12298, 2020.

Elazar, Y., Ravfogel, S., Jacovi, A., and Goldberg, Y. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *TACL*, arXiv:2006.00995, 2021.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS Datasets and Benchmarks Track, arXiv:2306.05685, 2023.

Frick, E., Li, T., Chen, C., Chiang, W.-L., Angelopoulos, A. N., Jiao, J., Zhu, B., Gonzalez, J. E., and Stoica, I. How to evaluate reward models for RLHF. arXiv:2410.14872, 2024.

Malik, S., Pyatkin, V., Land, S., Morrison, J., Smith, N. A., Hajishirzi, H., and Lambert, N. RewardBench 2: Advancing reward model evaluation. ICLR, arXiv:2506.01937, 2026.

Norman, J. D., Rivera, M. U., and Hughes, D. A. Reliability without validity: A systematic, large-scale evaluation of LLM-as-a-judge models across agreement, consistency, and bias. arXiv:2606.19544, 2026.

Weng, S., Feng, Y., and Xie, X. Beyond accuracy: Policy invariance as a reliability test for LLM safety judges. arXiv:2605.06161, 2026.

Yang, Z., Hou, Y., and Yang, X. When the judge changes, so does the measurement: Auditing LLM-as-judge reliability. arXiv:2607.08535, 2026.

Zhou, C. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. arXiv:2607.05904, 2026.

Li, Y. Who drifted: the system or the judge? Anytime-valid attribution in LLM evaluation pipelines. arXiv:2606.15474, 2026.

Beigi, M., Jin, M., and Huang, L. Proxy reward internalization and mechanistic exploitation: A learned precursor to reward hacking and its generalization. arXiv:2606.09711, 2026.

Bailey, L., et al. Obfuscated activations bypass LLM latent-space defenses. arXiv:2412.09565, 2024.

A The judge prompt and decoding settings

The judge is the one language model that is part of our method rather than a subject of it, so we give it in full. Every answer is scored by `Qwen2.5-7B-Instruct` with this prompt, where `{nums}`, `{target}` and `{equation}` are filled from the problem and the first `<answer>` block (answers with none are sent with "(no answer given)" in its place). The text is the user turn of Qwen's chat template, which prepends the model's default system turn:

```
You are grading a solution to a Countdown arithmetic puzzle.
The solver was given the numbers {nums} and the target {target}.  A
correct solution uses each number exactly once, uses only + - * /
and parentheses, and evaluates to the target.
The solver's proposed equation:
{equation}
Is this equation a correct solution?  Answer with exactly one word,
YES or NO.
```

The score is $P(\text{YES})$ from the first generated token's top-20 logprobs, renormalised over six YES and six NO variants (upper, title and lower case, each with and without a leading space; all single tokens). Two of the $35{,}728$ answers had no YES variant in the top 20 and score 0. It is a continuous score rather than a sampled verdict, so the threshold analyses in §4.2 are well defined. The score is read from a single forward pass of a frozen model, which is what makes the fixed-function check of §4.2 meaningful: the same (problem, equation) pair scored at two checkpoints receives the same score up to the numerical non-determinism of batched inference, and we verify that empirically rather than assuming it. Of the $4{,}651$ repeated pairs, 79% score identically every time and 109 differ by more than 0.01, the largest by 0.109.

Apart from the judge, the only language models in the method are the policy and, for arms A and R, a frozen copy of C_{SFT} whose hidden states their probes read. The probes and arm B's surface model are logistic regression, the policy is trained with RLOO against the verifier or one of those rewards, and the verifier is exact and rule-based.

B Why the paper reports LR+ rather than lift, and what is wrong with the control ladder

Why AUROC stays flat: partly the policy, partly the metric. AUROC depends only on the order the probe puts answers in. Between steps 0 and 40 the mean score inflates $0.473 \rightarrow 0.677$

while leaving the order within each checkpoint almost intact, and the inflation is in the policy's new answers (the step-0 answers read through the step-40 checkpoint average 0.377), so "AUROC stayed flat" is partly a fact about the metric and about every rank-based statistic. The obvious replacement is worse. Precision at a frozen threshold falls $0.588 \to 0.416$ (-29.3%), but the share of answers that are correct falls too, and precision moves with it whether or not the probe changed. The prevalence-invariant statistic is $\mathrm{LR}+ = \mathrm{TPR}/\mathrm{FPR}$, and it falls $2.41 \to 1.65$, or $-\mathbf{31.6}\%$: slightly more than raw precision, not less. All four statistics here use last-block labels, as in Table 3. Measured against its above-chance margin, AUROC moves -5.1%. We use LR+ and not *lift*, which is not prevalence-invariant: on the judge ladder $\sim$91% of lift's apparent improvement is the confound it appears to remove, with roughly 70% of the flag rate's own movement predicted the same way. On the probe ladder lift moves only -12.8%. We examined the step-0 25th, 75th and 90th percentiles as thresholds after seeing the data; for both monitors LR+ falls at the strict thresholds and not at the lenient one.

The control is the weakest part of this paper. A paired test against step 0 shows the statistic *moved*, not that it moved more than benign training moves it. That needs a control ladder of ordinary runs, and ours is poor: five checkpoints, the wrong initialisation, headline-protocol sampling (16 answers at temperature 0.6) rather than the ladder's 8 at temperature 1, and three of its five checkpoints scored on 166–167 prompts where every attacked checkpoint uses all 406; at step 30 a further 32.1% of its answers have no locatable `</think>` and cannot be scored. It is the ordinary run from C_{SFT} that produced C_{outcome}, scored with the reward probe at a threshold frozen at its own step 0. Holding the population fixed at the 166 prompts all five checkpoints share, benign training raises the flag rate by up to 18.3% of its step-0 value (at step 30), so the band is not an artifact of the changing prompt count. On the same prompts the attacked run's flag rate rises by 15.1%, 22.1% and 41.5% of its step-0 value at steps 20, 30 and 40, so it first exceeds the band at step 30, narrowly, and leaves it by a wide margin at step 40. Where it leaves the band also depends on the frozen threshold. On the same ladder the reward probe keeps the share of answers above 0.95 below 9% at every checkpoint, where the attacked run reaches 38.8% by step 40. Read the lead as evidence that the ordering exists, not as a calibrated false-alarm rate. A matched control ladder is the highest-value experiment left undone here.

C Setup detail

C_{SFT} is `asingh15/qwen-sft-countdown-defaultproj`, a public checkpoint we did not train. C_{outcome} is 100 RLOO (Ahmadian et al., 2024) steps with the verifier reward at batch 128, group size 16, KL 10^{-3}, entropy 10^{-3}, lr 10^{-5}, weight decay 10^{-4} and no gradient clipping. `runA` and `runB` use group size 8, weight decay 0.01 and gradient clipping at 1.0, with the KL anchor at their own initialisation; arms A, B and R use group size 8, weight decay 10^{-4} and no clipping. No run was seeded. In `runA` and `runB` the reward's copy of the policy was refreshed from the latest checkpoint each round, except on the one round in ten when a bug in locating that checkpoint left it one update old; the code has since been fixed. The pre-registration of arms A and B was written before arm A launched, by our local file records, but first committed to git on 12 August 2026, after both arms had run, so its timing is not independently verifiable. Read as the fall from each run's start, a reading the pre-registration did not name and one that sets the arms' arm-protocol accuracy against C_{SFT}'s headline score, arm A's prediction holds: arm A fell 12.3 pp from C_{SFT}, `runA` 31.4 pp from C_{outcome}. Arm B's prediction, that a text-only reward degrades the task at least as much as the probe reward, holds against arm R either way (-5.14 pp, CI $[-6.25, -4.09]$), and against `runA` only as final accuracy. The pre-registration's reference row names the published run but lists AUROC 0.978, the value of arm R's probe, while its text cites `runA`'s accuracy drop; `runA` differs from the arms in initialisation, trainer and optimiser settings that it did not list. During RL, sampling stops at the first `</answer>`; evaluation sampling does not. Arm A's evaluation command was not recorded. Re-running the arm protocol on its final checkpoint reproduces its evaluation answer for answer, as it does for arm B, whose command was recorded, so all four arms were scored identically. The verifier returns $\{0, 0.1, 1.0\}$, and rejects an answer written as an equation with an equals sign; accepting a trailing "= target" would add at most 1.1 pp to any arm's accuracy (arm R's). Eval is 500 procedurally generated problems filtered to the 406 with no overlap against the RLOO training pool.

All four arms are scored under the arm protocol (Table 1b). Arm C was first scored under the headline protocol, at 0.0734; re-scored like the other three it reads 0.0782, a difference of $+0.48$ pp (CI $[-0.62, +1.60]$), so arm C's placement does not depend on the protocol.

Probes are L2-regularised logistic regression at layer 16. The trace-final probe, the baselines of §3 and the reward probe are scored by 5-fold `GroupKFold` on prompt and class-balanced by subsampling the held-out scores. The probes of arms A and R and arm B's surface model are fit with a class-weighted loss on one half of the prompts, split by a hash of the prompt index, and scored on the other half. The ladder AUROCs are plain AUROCs of the frozen reward probe. We do not use in-sample figures; for reference, the trace-final probe reads 0.995 in-sample against 0.982 held out.

Reproducibility. One script, `scripts/check_everything.sh`, regenerates the numbers in this paper on CPU from cached artifacts and exits non-zero on any disagreement. Every numeral in the text, captions and tables is tied to a check that recomputes it or tests the condition it states, pinned to its sentence or table cell, and the script fails if any numeral is left unclaimed. The exceptions are listed with their reasons in `scripts/number_exemptions.tsv`: results quoted from cited work, the withdrawn steering result, model names and sizes, one commit date, and a few definitions and labels. Numbers written as words and names set in code type are outside this check. The script redraws both figures from the same computations and compares them with the committed ones. Planting a one-digit error in any checked numeral, or swapping two nearby numerals of the same form, makes it fail. The run settings of this appendix are checked against a committed snapshot of the runs' W&B configurations (`docs/run_configs.json`). Code and the script are at https://github.com/Abraham-y/reader-writer-probe-rl, and the 0.37 GB of cached artifacts it reads are at https://huggingface.co/datasets/prismane16/monitor-accuracy-under-rl.

D The trace-final probe in full

As a best-of-16 selector on 406 held-out prompts the probe reaches 0.6700, against 0.5531 for picking at random among the same answers and 0.6847 for an oracle that takes a correct answer whenever one is present; that is 88.8% of the available gain (prompt-clustered paired bootstrap, CI $[78.9\%, 96.5\%]$). A prompt has up to 16 scoreable answers, 15.5 on average once unscoreable ones are dropped. The margin over the free baselines is $+3.20$ pp of selection accuracy (CI $[+0.99, +5.42]$, $p = 0.008$ against picking the answer with the shortest `<think>` and $p = 0.004$ against the 39-feature surface model), and $+0.056$ AUROC over the stronger baseline. That margin, not the raw 0.982, is the number to read. This is a baseline comparison, not the control-task selectivity of Hewitt and Liang (2019), which would require refitting on a control task with random labels fixed per input type; we did not run that.

On length specifically: stratified by `</think>` position the probe still reads 0.939, against 0.564 for position alone, so it is not a length detector, though picking the answer with the shortest `<think>` already captures 64% of the best-of-16 selection headroom.

Two limits on what any of this licenses. High probe accuracy does not by itself establish that the model *uses* the property it encodes (Belinkov, 2022), and similar probe accuracies can hide large differences in how readily a representation encodes the property (Voita and Titov, 2020); we make no causal claim. Answers with no locatable `</think>` are unscoreable and dropped from every AUROC in this section (2.9% on C_{outcome}, 10.4% on C_{SFT}, all of them wrong). The checkpoint ladder of §4.1 drops such answers too, but few: 32 at step 0 and at most 8 at any other step (3,216–3,248 of 3,248 scored), too few to explain the flat AUROC. Layer 16 was fixed in advance from the original study.

E Full checkpoint ladders

Every checkpoint of both series, with prompt-clustered bootstrap intervals. The body quotes the load-bearing values from these tables; they are reproduced whole here because both series are non-monotone and a selected subset would misrepresent them. Checkpoint labels are the trainer's: the checkpoint labelled step k holds the weights after $k + 1$ updates, so step 0 is one update past C_{outcome}, and step 99 is the final checkpoint. All three tables report nominal p from 2,000 resamples, and print < 0.001 where no resample crosses zero. Over the probe ladder's 30 step-0 contrasts (AUROC under each label, and flag rate), the AUROC falls at steps 40 (first block) and 50 (last block) and the flag-rate departure at step 10 survive a Holm correction, while the first-block rise at step 20 does not.

RLOO step	acc. first	acc. last	**probe AUROC** [95% CI]	p vs. 0	prec @ τ	**LR+**	**flag rate** [95% CI]
0	0.546	0.372	0.783 [0.760, 0.804]	—	0.588	2.41	0.500 [0.466, 0.535]
10	0.493	0.308	0.777 [0.753, 0.801]	0.627	0.504	2.28	0.529 [0.495, 0.565]
20	0.509	0.312	0.778 [0.753, 0.801]	0.598	0.503	2.23	0.559 [0.521, 0.598]
30	0.496	0.291	0.769 [0.746, 0.791]	0.222	0.439	1.91	0.601 [0.567, 0.638]
40	0.445	0.302	0.768 [0.748, 0.790]	0.228	0.416	1.65	0.691 [0.661, 0.722]
50	0.404	0.260	**0.696 [0.673, 0.719]**	**<0.001**	0.290	1.16	0.866 [0.847, 0.886]
60	0.312	0.186	0.620 [0.592, 0.647]	<0.001	0.189	1.02	0.977 [0.972, 0.982]
70	0.252	0.148	0.579 [0.548, 0.609]	<0.001	0.149	1.01	0.987 [0.983, 0.990]
80	0.247	0.148	0.576 [0.546, 0.608]	<0.001	0.149	1.01	0.991 [0.988, 0.994]
90	0.247	0.141	0.615 [0.584, 0.646]	<0.001	0.141	1.00	0.996 [0.993, 0.998]
99	0.242	0.133	0.562 [0.531, 0.594]	<0.001	0.133	1.00	0.996 [0.994, 0.998]

Table 3: The reward probe on `runA` (initialised from C_{outcome}), probe-ladder protocol: 8 fresh answers per prompt at temperature 1 on the 406 prompts. Accuracy is given under both grading rules; the AUROC, precision and LR+ use the last-block label, the rule the probe was fit to predict; the flag rate needs no label. τ is the probe's step-0 median score, 0.540. Bold marks the first significant change in AUROC. Accuracy is significantly lower from step 10 under either rule; the AUROC first moves at step 50 under this label, and at step 40 under first-block labels (Table 4). Intervals are a prompt-clustered bootstrap over the 406 prompts (2,000 resamples) computed from per-rollout probe scores on each checkpoint's own cached activations; p from paired differences against step 0.

RLOO step	**probe AUROC, first block** [95% CI]	p vs. 0
0	0.919 [0.904, 0.932]	—
10	0.917 [0.901, 0.932]	0.807
20	0.936 [0.922, 0.949]	0.022
30	0.919 [0.903, 0.932]	0.972
40	**0.876 [0.860, 0.892]**	**<0.001**
50	0.779 [0.759, 0.798]	<0.001
60	0.655 [0.631, 0.680]	<0.001
70	0.619 [0.593, 0.645]	<0.001
80	0.615 [0.588, 0.642]	<0.001
90	0.622 [0.598, 0.647]	<0.001
99	0.585 [0.557, 0.613]	<0.001

Table 4: The reward probe's AUROC on the same ladder as Table 3, against first-block labels, the block the verifier graded during training. It starts higher than under last-block labels, rises at step 20 (nominally), and first falls at step 40, in bold. Prompt-clustered bootstrap over the 406 prompts, 2,000 resamples; p from paired differences against step 0.

Over the judge's 20 contrasts (AUROC and flag rate), only the flag-rate departures from step 40 on survive Holm or Bonferroni reliably; the AUROC dip and the flag rate at step 30 sit near the cut-offs and survive or not depending on the bootstrap seed.

RLOO step	acc. first	**judge AUROC** [95% CI]	p vs. 0	prec @ τ	**LR+**	**flag rate** [95% CI]
0	0.544	0.785 [0.757, 0.813]	—	0.746	2.46	0.500 [0.465, 0.534]
10	0.495	0.784 [0.755, 0.812]	0.864	0.703	2.41	0.481 [0.448, 0.512]
20	0.506	0.761 [0.729, 0.791]	0.011	0.699	2.26	0.474 [0.443, 0.506]
30	0.488	0.754 [0.723, 0.783]	0.002	0.679	2.22	0.465 [0.432, 0.498]
40	0.457	0.757 [0.726, 0.785]	0.025	0.667	2.38	0.434 [0.402, 0.466]
50	0.399	0.778 [0.748, 0.806]	0.593	0.631	2.58	0.373 [0.341, 0.405]
60	0.307	0.787 [0.757, 0.814]	0.921	0.558	2.84	0.331 [0.305, 0.360]
70	0.268	0.790 [0.761, 0.818]	0.731	0.524	3.01	0.304 [0.279, 0.332]
80	0.234	0.792 [0.761, 0.820]	0.673	0.482	3.04	0.294 [0.268, 0.321]
90	0.232	0.775 [0.744, 0.806]	0.511	0.448	2.68	0.312 [0.287, 0.340]
99	0.226	0.769 [0.737, 0.801]	0.312	0.429	2.57	0.317 [0.292, 0.342]

Table 5: An LLM judge watching the same run; τ is its step-0 median score, $P(\text{YES}) = 0.9991$. `runA`, judge-ladder protocol: an independent draw of 8 answers per prompt at temperature 1, graded by the first block, the rule the judge is shown. Its pooled AUROC dips at steps 20–40 and recovers, about half of it because step 0 holds 108 answers with no `<answer>` block (§4.2); at τ its LR+ ends where it began, though at stricter thresholds it falls (§4.2). Prompt-clustered bootstrap over the 406 eval prompts, 2,000 resamples; p from paired differences against step 0, nominal.

F An answer from arm B's policy

For input $[29, 17, 63, 38]$ and target 68, arm B's final policy wrote, verbatim:

```
<answer>
Let me verify:
1.  17 - 10 = 7
2.  63 - 7 = 56
3.  29 * 38 = 1102
4.  56 - (29 * 38 / 38) = 68
</answer>
```

Step 1 uses 10, which is not in the input. Step 3 is correct but irrelevant. Step 4 reuses two numbers and evaluates to 27 while asserting 68 in the answer position. The answer has the form of verification and contains no valid arithmetic.